CASE 03 · BUSINESS ANALYST PORTFOLIO PROJECT

Turning Reviews Into a Roadmap

An AI-assisted backlog prioritization case study, mined from real Google Play reviews of the ChatGPT mobile app.

Role simulated: Business Analyst / Product Analyst | Tools: Python (rule-based + AI-assisted theme mining), RICE prioritization, Jira

Source: Google Play Store Apps & Reviews dataset (public GitHub mirror, luminati-io)

169	20	10	7
reviews screened for this app	reviews tagged to a concrete theme	mentions of the #1 theme (reliability)	backlog stories, RICE-ranked

The business problem

A product team has a pile of app-store reviews and no structured way to turn them into sprint work. This project mines real reviews into a RICE-prioritized, Jira-importable backlog — the same shape of exercise a product-analyst-leaning BA role increasingly expects.

Method — and a deliberate honesty check

169 reviews were screened (low-and-mid star ratings, plus any longer review regardless of rating). Rule-based theme tagging was used instead of a black-box AI summary, for a specific reason: it's inspectable — every tagged review can be traced back to the exact phrase that triggered its theme, which is what you want when you're about to turn a theme into engineering work.

Worth saying plainly: this public sample is small (169 screened reviews for one app, only 20 tagged to a theme). Read the ranking below as a directional signal for how to structure a backlog, not a statistically powered study. The same method holds if you point it at a fuller export from the Play Console API — that's the honest caveat to lead with in an interview, not a weakness to hide.

Theme mining results

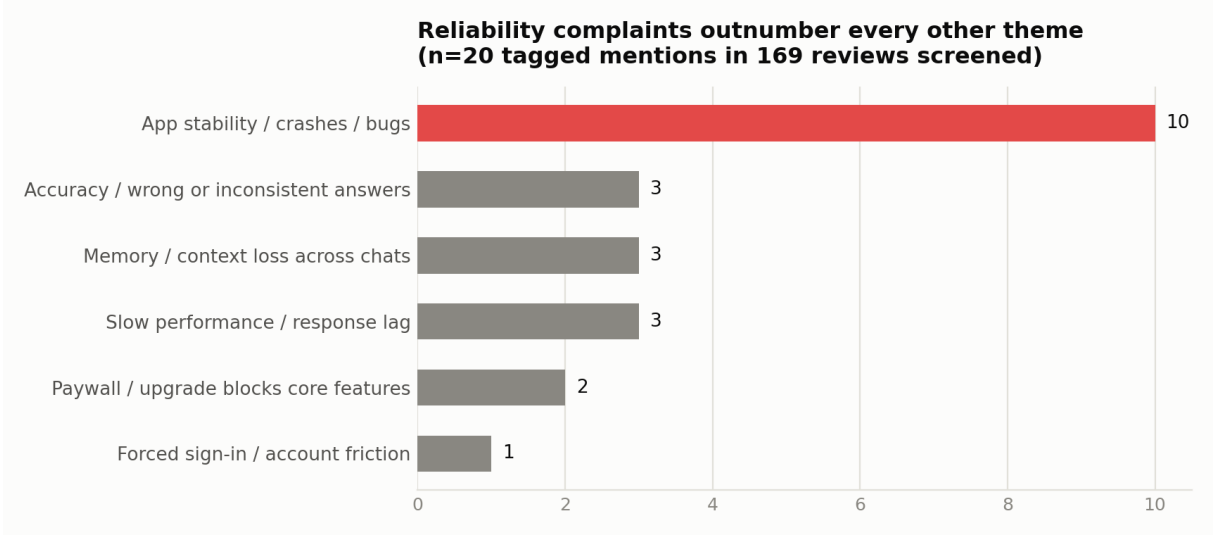


Fig. 1 — Rule-based theme frequency across screened reviews.

Backlog: epics, stories & acceptance criteria

Reliability & Bug Triage

Story (rank 1, RICE 7.2): Retry a failed message send without duplicating it

- A failed send shows a retry action inline, not a lost message.
- Retrying never creates a second copy of the same message in the thread.

Story (rank 2, RICE 6.4): Show a clear error state (network vs server vs input) instead of a silent failure

- Network, server, and input errors each show a distinct, human-readable message.
- No request fails silently with no visible state change.

Transparent Paywall & Usage Limits

Story (rank 3, RICE 3.4): Show which features require upgrading before the user attempts them, not after

- Paywalled actions (e.g. image generation) are visibly marked before the user starts the action.
- No action is allowed to run partway and then fail on a paywall the user couldn't see coming.

Conversation Memory & Consistency

Story (rank 4, RICE 2.1): Warn the user before a long chat's context window truncates

- A visible warning appears before older context is dropped from a long chat.
- The warning explains what will happen if the user continues (older messages forgotten).

Story (rank 6, RICE 1.2): Keep the fix to a corrected error from being repeated later in the same chat

- Once a user corrects the assistant within a chat, that correction persists for the rest of the session.

• QA regression case: correct an error, continue chatting 10+ turns, confirm the original error doesn't recur.

Frictionless Onboarding

Story (rank 5, RICE 1.7): Let a new user send their first message before requiring account sign-in

• A new install allows at least one message exchange before any sign-in prompt.

• Sign-in is requested at a natural save/continue point, not as a hard blocker on first launch.

Answer Accuracy & Trust Signals

Story (rank 7, RICE 1.1): Flag lower-confidence answers on factual / diagram-based questions

• Responses involving diagrams or cited facts carry a visible confidence/caveat indicator.

• The indicator links to a short explanation of why (e.g. "diagram generation is approximate").

RICE-ranked backlog

#	Story	Reach	Impact	Conf.	Effort	RICE	Sprint
1	Retry a failed message send without duplicating it	8	3	90%	3	7.2	S1
2	Show a clear error state (network vs server vs input…	8	2	80%	2	6.4	S1
3	Show which features require upgrading before the use…	4	2	85%	2	3.4	S1
4	Warn the user before a long chat's context window tr…	5	3	70%	5	2.1	S1
5	Let a new user send their first message before requi…	7	2	60%	5	1.7	S2
6	Keep the fix to a corrected error from being repeate…	5	2	60%	5	1.2	S2
7	Flag lower-confidence answers on factual / diagram-b…	6	3	50%	8	1.1	S2

RICE = (Reach × Impact × Confidence) / Effort. Reach/Impact/Effort are team-judgment inputs on a 1–3 / 1–10 style scale calibrated for this exercise; Confidence is a %. Full precision numbers are in rice_backlog.csv.

Sprint plan & simulated velocity

The top 4 stories by RICE score were committed to Sprint 1, the remaining 3 to Sprint 2. Execution is simulated for this exercise (no live team) — the chart below shows the shape of a velocity tracking view, with one story slipping in Sprint 1 as a realistic example of what that looks like.

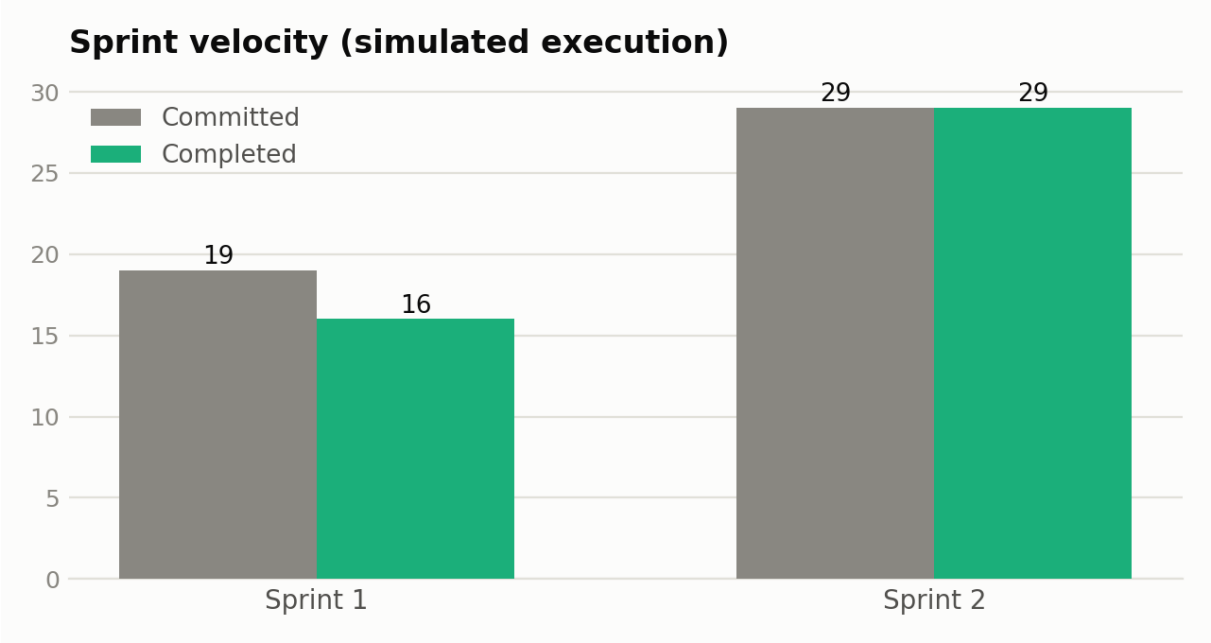


Fig. 2 — Committed vs. completed story points per sprint (simulated).

RESUME BULLET, READY TO USE

“Mined 169 real app-store reviews using rule-based text analysis to surface top user pain points; translated findings into a RICE-prioritized backlog of 7 user stories with acceptance criteria, packaged for direct import into Jira.”

What's in this package

File	What it is
run_analysis.py	Theme mining, RICE scoring, and chart generation — fully reproducible.
rice_backlog.csv	The full RICE-ranked backlog with all scoring inputs.
jira_import_backlog.csv	Same backlog, formatted for Jira's CSV importer — epics + stories, ready to load.
This report (PDF)	Epics, user stories, acceptance criteria, RICE table, and sprint plan.